

Local Two-Controller Battle & Machine Learning Mirror-Match Pilot

Review draft for the first two implementation phases

Project: Dice and Destiny v2

Prepared: August 1, 2026

Status: Documents first; no implementation or branch changes yet

Primary combatant: Blade Warden (`blade\_warden`)

Decision gate: Owner review and approval before Phase 1 begins

Proposed outcome: First make the local Go battle authority support two equal externally controlled seats using only existing content. Then train the first machine-learning policy instances to play Blade Warden mirror matches through that exact authority boundary.

1. Executive direction

The project should proceed in two deliberately narrow phases. Phase 1 creates a controller-neutral, local, two-seat battle mode. Phase 2 creates the first learning pilot in which two policy instances play the same existing Blade Warden combatant against each other. This sequence separates battle-engine correctness from machine-learning uncertainty and creates useful test infrastructure before model training begins.

The work is not an online multiplayer project. There is no matchmaking, lobby, account, network transport, remote hosting, or production PvP interface in scope. Both seats, the authority, the training environment, and the model processes run on one development computer.

The work also creates no new creatures, cards, dice, statuses, abilities, elements, team systems, or progression content. The existing rules and deliberately limited content remain the controlled experimental baseline.

2. Immediate goals and boundaries

Area	In scope now	Explicitly outside these two phases
Battle topology	One existing combatant in Seat A versus one existing combatant in Seat B	Four-creature teams, switching, bench state, recruitment
Control	Two local external controllers; human test driver, scripted policy, heuristic, or learned policy	Network clients, matchmaking, remote sessions

Area	In scope now	Explicitly outside these two phases
Content	Blade Warden mirror match using current YAML definitions	New or redesigned creatures and balance changes
Rules	Both seats use the complete real roll, keep, reroll, card, ability, target, defense, and reaction flow	D100 reveal shortcuts as the equal-seat balance oracle
ML	First reproducible self-play pilot and evaluation report	Production-ready universal roster policy
Delivery	Reviewable documents, then a dedicated implementation branch after approval	Implementation before owner review

Naming clarification: The existing sword-oriented player combatant is Blade Warden, with the stable content ID ``blade_warden``. That is the combatant used in both acceptance phases.

3. Current technical baseline

The repository already contains most of the foundations that make this practical:

- **Portable Go authority.** Commands enter the authority as typed JSON and produce validated events, pending input, and viewer-safe snapshots.
- **Automatic progression boundary.** The engine advances through automatic work until an external actor genuinely needs input.
- **Viewer filtering.** Snapshot construction already distinguishes what a particular actor may see, which is essential for honest model observations.
- **Determinism.** Reproducible random state and named random streams exist for combat dice, card draws, AI selection, damage selection, and other domains.
- **Cloneable state.** Battle state can be deep-cloned, enabling branching tests and future search agents.
- **Full-battle proof.** A deterministic four-round Blade Warden versus Venom Goblin authority test already exercises the real command path.

The current settled implementation is nevertheless asymmetric. It discovers a single human actor, opens input windows for that actor, and automatically resolves AI-controlled participants. The current Venom Goblin offensive policy selects from a YAML D100 chart and installs a predetermined reveal profile; it is a production PvE policy, not an equal-player balance standard.

Relevant implementation hotspots

`dice-and-destiny-server/internal/battle/authority.go`

`dice-and-destiny-server/internal/battle/engine/engine.go`

dice-and-destiny-server/internal/battle/engine/settled_engine.go
dice-and-destiny-server/internal/battle/participant_assembler.go
dice-and-destiny-server/internal/battle/snapshot/snapshot.go
dice-and-destiny-server/internal/battle/random/random.go
dice-and-destiny-server/internal/battle/full_battle_authority_test.go

4. Target controller architecture

Combatant identity and decision ownership should become separate concepts. A combatant definition describes cards, dice, abilities, resources, health, and status behavior. A controller describes who or what chooses a legal action for a runtime seat.

Controller	Purpose in the two-phase plan
External controller	A local human driver, test script, or model process receives observations and submits actions.
D100 controller	Preserves the existing Venom Goblin PvE behavior as a baseline and fallback; it is not used for the equal Blade Warden mirror match.
Random legal controller	Validates action enumeration and establishes a minimum-skill benchmark.
Heuristic controller	Provides a debuggable baseline before reinforcement learning.
Learned policy controller	Scores the legal actions exposed by the authority and returns one action index.

Authority rule: No controller may mutate battle state directly. A controller selects from fully formed legal commands; the Go authority validates and applies the selected command through the normal rules path.

5. Phase 1 — Local two-controller battle

Phase 1 proves that two equal external seats can complete a real battle on one computer. The acceptance matchup is Blade Warden versus Blade Warden. The phase should favor a server-level local driver and automated tests over a polished Godot hot-seat interface because the immediate consumer is the machine-learning environment.

5.1 Required work packages

1. **Separate combatant from controller.** Allow an existing combatant definition to be assigned to an external seat independently of its authored default controller. Preserve controller defaults as convenience, not as a validation lock.
2. **Create symmetric battle setup.** Start a battle with Seat A and Seat B as peers. Both use existing Blade Warden content, their own instantiated cards, their own dice state, and their own private viewer identity.
3. **Replace the single-human assumption.** Represent pending input by actor or by required-actor set. Automatic progression must stop whenever either external seat has a legal decision.
4. **Preserve hidden simultaneous planning.** Seat B must not observe Seat A's hidden offensive commitment before Seat B commits. Reveal occurs only when the rules say both commitments are complete.
5. **Define reaction priority.** Specify and test who receives the first reaction opportunity, how priority passes, when a reaction round ends, and how seat advantage is measured.
6. **Enumerate complete legal actions.** Return concrete legal commands with current pending-input IDs, checkpoints, targets, dice subsets, card instances, and other required payload fields. Do not expose only command names.
7. **Provide a local two-seat driver.** Offer a developer-facing driver that can inspect each seat's viewer-safe state and submit a legal choice. A CLI or focused local tool is sufficient; no networking is required.
8. **Protect existing behavior.** Keep existing authority, deterministic battle, recovery, snapshot, and Godot verification tests passing. The D100 path remains available for current PvE behavior.

5.2 Phase 1 interaction contract

Operation	Required behavior
Reset	Create a fresh Blade Warden mirror battle from existing YAML content with a reproducible seed.
Observe seat	Return only the snapshot, event information, and history legally visible to that seat.
List legal actions	Return complete legal action candidates for the seat that currently owns input.
Submit	Accept one candidate as a normal authority command; reject stale, illegal, or wrong-seat commands.

Operation	Required behavior
Advance	Resolve automatic rules until one or both external seats require another decision, or the battle ends.
Record	Capture seed, content version, selected actions, terminal result, and replay/debug identifiers.

5.3 Phase 1 acceptance criteria

- ☐ A battle can start with two distinct externally controlled Blade Warden instances.
- ☐ Both seats use the complete player decision flow; neither uses the D100 offensive shortcut.
- ☐ Seat A and Seat B receive independent viewer-safe observations.
- ☐ Hidden planning information does not leak before reveal.
- ☐ Each pending input exposes at least one complete legal action and the authority accepts every enumerated choice when submitted immediately.
- ☐ Stale, wrong-seat, and fabricated commands are rejected.
- ☐ Reaction priority is explicit, deterministic under a seed, and covered by tests.
- ☐ A deterministic scripted mirror battle reaches a valid victory, defeat, or draw through the normal authority path.
- ☐ A local developer can manually drive both seats without starting Godot or a network service.
- ☐ The standard Go and Godot authority verification suites continue to pass.

5.4 Branch and approval gate

No branch is created as part of this documentation task. After approval, create a dedicated branch using the repository's required prefix. Recommended branch name:

`codex/local-two-controller-battle`

Phase 1 should finish, pass verification, and receive review before model-training code is introduced. Phase 2 can then continue on a new branch based on the accepted Phase 1 baseline or on the same branch only if the owner explicitly prefers one long-lived experimental branch.

6. Phase 2 — First machine-learning mirror match

Phase 2 builds the smallest credible learning experiment: two policy instances play Blade Warden against Blade Warden through the Phase 1 controller interface. The initial objective is not to prove that Blade Warden is balanced. It is to prove that training is reproducible, that learned agents can finish legal battles, and that the resulting behavior can be measured and replayed.

6.1 Model choice

Use a small reinforcement-learning policy rather than a large language model. The battle state and actions are already structured; an LLM would add memory, latency, packaging, and legal-action problems without a corresponding benefit. Begin with a modest masked multilayer policy and CPU training. Maskable PPO is a reasonable first algorithm once the environment passes random and heuristic validation.

Two seats may initially use two instances of the same policy architecture. They can share weights for symmetric self-play while keeping independent episode state. Later experiments may compare independently seeded policies or past checkpoints, but that is not required for the first pilot.

6.2 Learning environment contract

Element	First-pilot decision
Environment	Long-running local Go simulator using the real engine; no Godot process and no subprocess launch per action.
Observation	Honest viewer-safe battle state plus necessary public history; no shuffled-card or hidden-commitment leakage.
Action	Index into a padded list of complete legal command candidates, with invalid slots masked.
Reward	+1 victory, -1 defeat, 0 draw. Add shaping only after proving it does not change the intended objective.
Episode end	Authority reports terminal battle status or a defined safety truncation prevents an infinite episode.
Randomness	Reproducible per-episode seeds and recorded content/model versions.
Parallelism	Multiple independent local environments; benchmark before selecting the worker count.

6.3 Build and training sequence

9. **Validate with random legal play.** Run large batches without crashes, illegal commands, stale input, or nonterminating battles.
10. **Add a transparent heuristic baseline.** The learned model needs something stronger than random play for comparison and debugging.

11. **Wrap as a vectorized training environment.** Batch observations, legal masks, rewards, terminal flags, and diagnostic metrics.
12. **Train from several seeds.** A single successful run is not reliable evidence of learning.
13. **Evaluate out of sample.** Hold back random seeds and compare both seat assignments.
14. **Replay surprising battles.** Record selected commands so diagnosis never depends on rerunning floating-point inference exactly.

6.4 First experiment design

Experiment	Purpose	Minimum output
Random vs random	Environment and seat-symmetry smoke test	Completion rate, invalid-action rate, seat win rate, duration
Heuristic vs random	Confirm that stronger decisions produce measurable advantage	Win rate with confidence interval and representative replays
Policy vs random	Prove the policy learned beyond chance	Held-out win rate across multiple training seeds
Policy vs heuristic	Establish whether the first policy is practically competent	Cross-play matrix and failure examples
Policy mirror	Detect seat advantage and policy instability	Seat-swapped results on matched battle seeds

6.5 Hardware expectation

The M3 Max is sufficient for the first pilot. The Go simulations, random/heuristic policies, search prototypes, and a small PPO policy are primarily CPU workloads. Begin on CPU, benchmark local parallel workers, and test Apple's MPS backend only if policy-network training becomes the bottleneck. Special hardware is not a prerequisite.

6.6 Phase 2 acceptance criteria

- ☐ Two model-controlled policy instances can register as the two local external seats.
- ☐ The model can select only from authoritative legal actions; invalid-action rate is zero outside deliberate negative tests.
- ☐ Training can be reproduced from recorded configuration, code revision, content version, and random seed.
- ☐ At least three training seeds complete and produce saved checkpoints and learning curves.
- ☐ Evaluation includes random, heuristic, seat-swapped mirror, and held-out-seed comparisons.
- ☐ At least 1,000 evaluation battles are used for the initial report; ambiguous findings can be confirmed with 10,000 or adaptive additional samples.

- ☐ The report includes confidence intervals, battle duration, remaining health, action frequencies, and representative replay traces.
- ☐ No claim that the creature is balanced is made solely from self-play convergence or a nominal 50 percent mirror result.

7. Measurement and interpretation

A mirror match is primarily a test of seat symmetry, controller quality, and rules integrity. If identical policies and identical combatants are swapped between seats, the long-run result should approach 50 percent unless initiative, reaction priority, or another seat-specific rule creates an advantage. That finding does not by itself determine roster balance because the matchup contains only one combatant.

Metric	Why it matters
Seat A / Seat B win rate	Detects first-action, reaction-priority, observation, or implementation bias.
Draw and truncation rate	Reveals stalemates and incomplete terminal rules.
Battle duration	Tracks pace and whether policies learn to delay indefinitely.
Remaining card-health	Shows margin of victory, not only binary outcome.
Damage and prevention per round	Separates offensive strength from defensive reliability.
Action distribution	Exposes a dominant ability, card, reroll pattern, or pass behavior.
Dead turns	Measures turns with no meaningful progress and possible reward-design problems.
Seed sensitivity	Distinguishes stable strategy from lucky training or evaluation.
Inference and simulation time	Confirms the model and environment remain practical for development and future shipping.

8. Delivery sequence and review gates

15. **Document approval.** Confirm scope, Blade Warden mirror baseline, local driver form, and proposed branch strategy.
16. **Phase 1 branch.** Create ``codex/local-two-controller-battle`` only after approval.
17. **Engine refactor.** Deliver controller neutrality, two-seat input, hidden information, legal action enumeration, and local driver.
18. **Phase 1 verification.** Run server tests and the required Godot authority verification through ``./scripts/godot.sh``. Review the scripted mirror battle.
19. **Phase 2 kickoff.** Freeze the Phase 1 environment contract before encoding observations and actions for training.

20. **Baseline report.** Publish random and heuristic results before neural training.
21. **Learning pilot report.** Publish multi-seed policy results and request direction before expanding the game or roster.

9. Principal risks within the two phases

Risk	Mitigation in this plan
Hidden information leaks	Build every observation through viewer-safe snapshot rules and add explicit negative tests.
Legal-action drift	Generate complete actions from authority state and immediately validate enumerated actions in tests.
Self-play blind spots	Evaluate against random and heuristic opponents and retain multiple training seeds.
Reward hacking	Start with terminal win/loss reward and treat secondary metrics as diagnostics, not objectives.
Seat advantage	Use matched random seeds, swap seat assignments, and report reaction priority explicitly.
Model tied to transient IDs	Encode mechanics and stable content features where feasible; version observation and action schemas.
Premature content balancing	Do not change Blade Warden during the infrastructure pilot. First measure engine and policy behavior.

10. Items for owner review

- ☐ Approve Blade Warden versus Blade Warden as both the Phase 1 acceptance battle and Phase 2 learning matchup.
- ☐ Approve a server-level local two-seat driver rather than a polished Godot hot-seat interface for Phase 1.
- ☐ Approve retaining the Venom Goblin D100 controller as a separate PvE baseline and fallback.
- ☐ Approve the recommended Phase 1 branch name `codex/local-two-controller-battle`.
- ☐ Approve Phase 1 completion and review as a gate before any model-training implementation begins.

Appendix A — Source references

Local code and design sources reviewed for this plan:

AGENTS.md

dice-and-destiny-server/internal/battle/README.md
dice-and-destiny-server/internal/battle/authority.go
dice-and-destiny-server/internal/battle/engine/settled_engine.go
dice-and-destiny-server/internal/battle/snapshot/snapshot.go
dice-and-destiny-server/internal/battle/random/random.go
dice-and-destiny-server/internal/battle/full_battle_authority_test.go
docs/game-design/dice-and-destiny-creature-system-working-design.docx

External implementation references:

[Gymnasium custom environments](https://gymnasium.farama.org/introduction/create_custom_env/) — https://gymnasium.farama.org/introduction/create_custom_env/

[Gymnasium vector environments](https://gymnasium.farama.org/api/vector/) — <https://gymnasium.farama.org/api/vector/>

[Stable Baselines3 Maskable PPO](https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html) — https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html

[Stable Baselines3 PPO](https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html) — <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>